

УНИВЕРЗИТЕТ У БЕОГРАДУ
МАТЕМАТИЧКИ ФАКУЛТЕТ



Марко Г. Лазаревић

ЕФИКАСНИ АЛГОРИТМИ ЗА ДИНАМИЧКЕ
ГРАФОВЕ ЗАСНОВАНИ НА
ХЕШ-ИНДЕКСИРАНИМ БЛОКОВИМА
СУСЕДСТВА

мастер рад

Београд, 2026.

Ментор:

др Весна МАРИНКОВИЋ, ванредни професор
Универзитет у Београду, Математички факултет

Чланови комисије:

др Данијела СИМИЋ, доцент
Универзитет у Београду, Математички факултет

др Мирко СПАСИЋ, доцент
Универзитет у Београду, Математички факултет

Датум одбране:

Наслов мастер рада: Ефикасни алгоритми за динамичке графове засновани на хеш-индексираним блоковима суседства

Резиме: Резиме

Кључне речи: графови

Садржај

1	Увод	1
2	Статички и динамички графови	3
2.1	Граф	3
2.2	Динамички граф	7
2.3	Математички модел динамичких графова	8
2.4	Динамички графови у моделу тока података	9
3	Имплементациони модел и структура	12
3.1	Ограничења формалног модела и поједностављен модел	12
3.2	Постојећи поједностављени модели	13
3.3	DHB структура	14
3.4	Операције и сложености	21
4	Алгоритми	23
4.1	Динамичка повезаност	23
4.2	Динамичко минимално разапињуће стабло	26
4.3	Динамичко максимално тежинско упаривање - Суитор алгоритам	28
5	Примене динамичких графова кроз примере	32
6	Закључак	33
	Библиографија	34

Глава 1

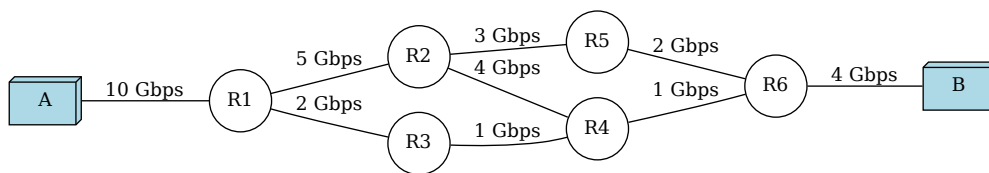
Увод

У поглављу 2 биће објашњени основни појмови везани за статичке и динамичке графове, као и различити типови динамичких графова.

Графови представљају једну од основних структура података у рачунарству. Једноставност ове апстрактне структуре омогућава велику изражајност и погодност за моделовање различитих система, од различитих типова мрежа и мапа до односа међу објектима. У пракси се граф представља конкретним структурама података, као што су листе суседства, матрице суседства или многи компресовани формати за ретке графове [14]. Избор репрезентације значајно утиче на ефикасност алгоритама, како у погледу временске, тако и просторне сложености. Већина стандардних алгоритама и структура за рад са графовима полази од претпоставке да је граф статички, односно да се скуп чворова и грана не мења током извршавања алгоритама. Насупрот томе, многи реални системи подразумевају промене током времена, где се чворови и гране често додају, уклањају или мењају [9]. Овакви системи захтевају проширење досадашњих погледа на графове.

Узмимо на пример комуникацију у рачунарској мрежи, која се природно може представити графом (слика 1.1). Рутери и везе између њих могу се појављивати и нестати услед различитих фактора као што су кварови, одржавање или промене у мрежној топологији. Иако се структурне промене у мрежи (додавање или уклањање чворова и грана) могу јављати релативно ретко, уколико се тежинама грана моделују динамичке величине, као што је тренутни пропусни опсег везе, и вредности тежина грана се могу мењати континуирано током времена. У таквом моделу, одржавање ажурних информација о стању мреже, укључујући оптималне путање рутирања и друга

релевантна својства, представља значајан алгоритамски изазов.



Слика 1.1: Неусмерена мрежа рутера између два рачунара

У таквим динамичким окружењима, алгоритми који раде на статичким графовима нису довољни, јер не узимају у обзир промене које се дешавају током времена и могу довести до нетачних резултата. Поновна анализа комплетне мреже након сваке промене постаје рачунски неизводљива, због чега су неопходни алгоритми и структуре података који се могу ефикасно прилагођавати изменама. Ови приступи омогућавају одржавање ажурних информација о мрежи, брзу реакцију на промене, као и ефикасно решавање задатака као што су праћење мреже, откривање аномалија и оптимизација протока података у окружењима са високим степеном динамике [9].

У овом раду ћемо се бавити наведеним изазовима, кроз анализу алгоритама и структура података за динамичке графове, са циљем проналажења ефикасних решења за проблеме који настају у оваквим променљивим окружењима.

Формална дефиниција динамичког графа биће наведена касније, али интуитивно, динамички граф је граф проширен временском димензијом у којој се мења његова структура. Ове промене могу бити узроковане различитим факторима, зависно од конкретне примене. Динамички графови представљају погодан формализам за моделовање и анализу система који се мењају током времена. Увођењем временског аспекта доводи се у питање не само како ефикасно представити и манипулисати графом, већ и како одржавати информације о његовој структури и својствима.

Глава 2

Статички и динамички графови

Како бисмо могли прецизније да дефинишемо динамичке графове, прво ћемо дати стандардну математичку дефиницију графа, а затим ћемо се фокусирати на динамичке графове, њихове карактеристике и разлике у односу на стандардне графове.

2.1 Граф

Граф G је уређени пар (V, E) који се састоји од скупа V чије елементе називамо *чворовима*, и скупа E чији су елементи парови чворова из V , које називамо *гранама* графа [13].

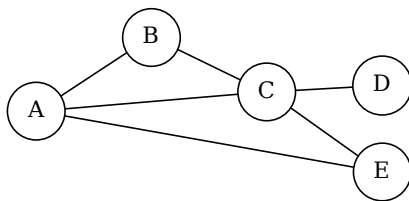
Уколико гранама графа G доделимо усмерење такав граф називамо **усмерени граф**, а у супротном за граф кажемо да је **неусмерен**. Неусмерену грану између чворова u и v означавамо паром $\{u, v\}$, док усмерену означавамо паром (u, v) , при чему је u почетни чвор, а v крајњи чвор те усмерене гране [13].

Графу $G = (V, E)$ можемо доделити функцију $g : E \rightarrow \mathbb{R}$, која свакој грани придружује реалан број који називамо **тежином** дате гране. На тај начин добијамо **тежински граф** $G' = (V, E, g)$. Граф чије гране немају тежину називамо **нетежинским графом** [13]. Неусмерени графови се могу представити помоћу усмерених заменом сваке неусмерене гране $\{u, v\}$ са две усмерене гране (u, v) и (v, u) . Нетежински графови се могу посматрати као тежински графови чије гране имају једнаку тежину (најчешће 1).

У случају неусмерених графова, кажемо да грана спаја два чвора којима је одређена, а за чворове те гране кажемо да су суседни. За неусмерену грану

$\{u, v\}$ која спаја чворове u и v кажемо да је инцидентна са тим чворовима, а чворове u и v називамо крајевима те гране. За гране инцидентне са истим чвором кажемо да су суседне. У случају усмерених графова, за усмерену грану (u, v) кажемо да излази, односно да је излазно инцидентна, из чвора u и улази, односно улазно инцидентна, у чвор v и да је чвор v сусед чвора u . Чвор u не мора бити сусед чвора v уколико не постоји ни једна грана усмерена од v ка u . [13].

Пример 1. На слици 2.1 је приказан неусмерени граф чији су чворови елементи скупа $V = \{A, B, C, D, E\}$, а гране елементи скупа $E = \{\{A, B\}, \{A, C\}, \{B, C\}, \{C, D\}, \{C, E\}, \{E, A\}\}$. Приметимо да редослед чворова у пару који представља грану није битан, па је тако, на пример, $\{A, B\}$ и $\{B, A\}$ иста грана.

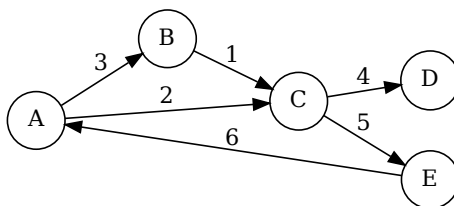


Слика 2.1: Неусмерени нетежински граф са пет чворова и шест грана

Пример 2. У примеру 1, усмерење грана није назначено, али ако узмемо да су гране усмерене од првог ка другом чвору у пару, тада је, на пример, грана (A, B) усмерена од A ка B . Уколико би се сматрало да су гране неусмерене, онда би грана (A, B) била и усмерена од A ка B и од B ка A . Такође графу G можемо доделити и функцију

$$g : (A, B) \mapsto 3, (A, C) \mapsto 2, (B, C) \mapsto 1, (C, D) \mapsto 4, (C, E) \mapsto 5, (E, A) \mapsto 6$$

која гранама придружује тежине, чиме добијамо тежински граф. На слици 2.2 је дат пример усмереног тежинског графа са пет чворова и шест грана.



Слика 2.2: Усмерени тежински граф са пет чворова и шест грана

Појам **степен чвора** представља једно од основних својстава чворова у графу. У неусмереном графу, степен чвора представља број грана инцидентних са тим чвором. У усмереном графу разликују се улазни степен (број грана које улазе у чвор) и излазни степен (број грана које излазе из чвора) [13].

Редак граф (енгл. *sparse graph*) је граф који има значајно мање грана од максималног броја грана који би могао да има. Специјални случај ретких графова су они графови чији је број суседа чворова ограничен неком (малом) константом [14].

У даљем тексту подразумеваћемо да се ради о усмереним, тежинским, ретким графовима, осим ако није другачије назначено.

Основни појмови и својства графова

Шетња у графу $G = (V, E)$ је низ облика $(v_1, e_1, v_2, e_2, \dots, e_{k-1}, v_k)$ чији су чланови наизменично чворови и гране тог графа такви да је грана e_i инцидентна са чворовима v_i и v_{i+1} ($1 \leq i \leq k-1$). Кажемо да шетња повезује чворове v_1 и v_k . Шетња је затворена ако важи $v_1 = v_k$, а у супротном је отворена. Дужина шетње једнака је броју грана које садржи. Неформално, кажемо да шетња пролази чворовима и гранама које садржи [13].

Стаза у графу $G = (V, E)$ је шетња која сваком граном пролази највише једанпут (произвољним чвором може проћи и више пута) [13].

Пут у графу је шетња која сваким чвором пролази највише једанпут [13].

Пут у усмереном графу $G = (V, E)$ (или усмерени пут) одређен је низом различитих чворова $(v_1, v_2, \dots, v_k)$ таквим да важи $(v_i, v_{i+1}) \in E$, за $1 \leq i \leq k-1$ [13].

Циклус (или затворена проста стаза) је пут код кога се почетни и крајњи чвор поклапају ($v_0 = v_k$), док су сви остали чворови међусобно различити. Дужина циклуса представља број грана које га чине [13].

За разумевање глобалне структуре графа кључан је појам повезаности. Неусмерени граф је **повезан** ако за било која два његова чвора постоји пут који их спаја. За усмерене графове разликујемо појам **јакe повезаности** (постоји усмерени пут између свака два чвора у оба смера) и **слабе повезаности** (граф постаје повезан када се уклоне усмерења са грана). Максимални повезани подграфови називају се **компоненте повезаности** графа [13].

Подграф графа $G = (V, E)$ је граф $H = (V_H, E_H)$ такав да је $V_H \subseteq V$ и $E_H \subseteq E$, при чему свака грана из E_H спаја чворове који припадају скупу V_H [13].

Стабло је повезан граф који не садржи циклусе. Граф који нема циклуса, а састоји се од више дизјунктних компоненти од којих је свака стабло, назива се **шума** [13].

Разапињуће стабло (енгл. *spanning tree*) графа $G = (V, E)$ је његов подграф $T = (V_T, E_T)$ који је стабло, при чему важи да је $V_T = V$. Уколико је граф G неповезан, тада се скуп разапињућих стабала његових компоненти повезаности назива **разапињућа шума** (енгл. *spanning forest*) [13].

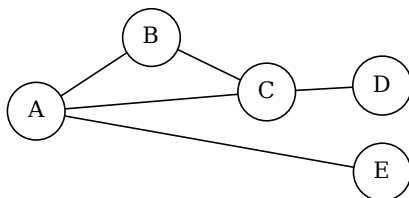
Нека је $G = (V, E, g)$ тежински граф. **Тежина подграфа** $H = (V_H, E_H)$ је сума тежина свих грана које му припадају, односно $W(H) = \sum_{e \in E_H} g(e)$. **Минимално разапињуће стабло** (односно шума) је оно разапињуће стабло T графа G чија је укупна тежина $W(T)$ минимална међу свим могућим разапињућим стаблима датог графа [13].

Упаривање у графу (енгл. *matching*) M у графу $G = (V, E)$ је подскуп грана $M \subseteq E$ са својством да никоје две гране из M не деле заједнички крајњи чвор. Чвор $v \in V$ је **упарен** уколико је инцидентан са неком граном из скупа M , а у супротном кажемо да је чвор **слободан**. Упаривање M је максимално уколико не постоји грана која се може додати у M а да M и даље остане упаривање [1].

У контексту тежинских графова, **максимално тежинско упаривање** (енгл. *Maximum Weight Matching*) је оно упаривање M чија је укупна сума тежина грана максимална [1].

2.2 Динамички граф

Према стандардној дефиницији графа, граф је статички објекат који се не мења током времена. По тој дефиницији, свака промена у структури графа би значила да се ради о новом графу. Тако је на пример граф $G' = (V', E')$ (слика 2.3) добијен уклањањем гране (C, E) из графа $G = (V, E)$ различит граф од G , јер су им скупови грана различити. Како бисмо ово превазишли, потребан нам је модел која ће омогућити праћење промена у графу током времена. Такав модел називамо **динамички граф**.



Слика 2.3: Неусмерен граф са пет чворова и уклоњеном граном (C, E)

Врсте динамичких графова

Динамичке графове можемо поделити на основу промена које се дешавају у њима. За тежински граф $G = (V, E, g)$, елементи који се могу мењати током времена су: скуп чворова V , скуп грана E и функција g која гранама придружује тежине. На основу тога, можемо издвојити следеће врсте динамичких графова [6]:

- **Чвор-динамички граф** (енгл. *vertex-dynamic graph*) је граф чији се скуп чворова V мења током времена. У овом случају, неки чворови могу бити додати или уклоњени. Када се чвор уклони, гране које су инцидентне са тим чвором такође се уклањају.
- **Грана-динамички граф** (енгл. *edge-dynamic graph*) је граф чији се скуп грана E мења током времена. У овом случају, гране могу бити додате или уклоњене из графа.

- **Тежински динамички граф** (енгл. *weight-dynamic graph*) је граф чија се функција g која гранама придружује тежине мења током времена.
- **Потпуно динамички граф** (енгл. *fully dynamic graph*) је граф у којем се све од наведених промена могу дешавати током времена, односно и скуп чворова V , и скуп грана E , и функција g која гранама придружује тежине могу се мењати током времена.

Други начин класификације динамичких графова је на основу начина на који се дешавају промене. На тај начин можемо издвојити следеће врсте динамичких графова [3]:

- **Инкрементални динамички граф** је граф код којег се чворови или гране само додају у граф G , али се не уклањају.
- **Декрементални динамички граф** је граф код којег се чворови или гране само уклањају из графа G , али се не додају.
- **Потпуно динамички граф** је граф код којег се чворови или гране могу и додавати и уклањати из графа G .
- **Динамички граф у моделу тока података** је граф код којег се структура мења током времена путем низа узастопних ажурирања (енгл. *data stream*), при чему се чворови и гране додају или уклањају један по један, а обрада се врши секвенцијално.

2.3 Математички модел динамичких графова

Прецизан модел динамичког графа описаћемо математичком структуром. Динамички граф $\mathcal{G}$ је уређена тројка $(\mathcal{V}, \mathcal{E}, \mathcal{T})$, где је $\mathcal{V} = \{V(t), t \in \mathcal{T}\}$ колекција скупова чворова током времена, $\mathcal{E} = \{E(t), t \in \mathcal{T}\}$ колекција скупова грана током времена, а $\mathcal{T}$ временски интервал. За сваки $t \in \mathcal{T}$, дефинишемо **верзију графа** (енгл. *snapshot*) $G_t = (V(t), E(t))$, односно статички граф који представља стање динамичког графа $\mathcal{G}$ у тренутку t [2]. Лако се види да овако дефинисан динамички граф подржава све врсте динамичких графова које смо раније навели, јер се скуп чворова $V(t)$ и скуп грана $E(t)$ могу мењати током времена. Ову дефиницију можемо проширити додавањем функције $g_t = \{g(t), t \in \mathcal{T}\}$ која сваком стању динамичког графа придружује

функцију тежине у тренутку t , чиме добијамо тежински динамички граф $\mathcal{G} = (\mathcal{V}, \mathcal{E}, g_t, \mathcal{T})$.

Иако математички прецизан, овако дефинисан модел динамичког графа није увек погодан за рачунску и алгоритамску примену. Наиме, у многим случајевима, системи који се моделују динамичким графом могу се мењати и неколико пута у секунди, што би захтевало да се комплетна структура графа чува и обрађује за сваки тренутак времена, што је рачунски неизводљиво. Уместо да памтимо комплетну структуру графа за сваки тренутак времена, можемо памтити секвенцу промена које су се десиле у току времена. Такав модел биће представљен у наредном поглављу.

2.4 Динамички графови у моделу тока података

Ток података (енгл. *data stream*) у општем смислу представља секвенцу дигитално кодираних сигнала који се користе за пренос информација [12]. У контексту алгоритама и обраде података, овај појам се апстрахује као низ елемената који пристижу секвенцијално, елемент по елемент.

Улазни ток

$$S = \langle a_1, a_2, \dots \rangle$$

описује еволуцију функције стања

$$A : \{1, 2, \dots, N\} \rightarrow \mathbb{R}.$$

Нека је са $A[i]$ означено стање функције A након обраде првих i елемената тока. Модели се разликују по томе како елементи a_i описују промене функције A [12].

У раду [12] издвајају се следећи основни модели:

- **Модел временских серија** (енгл. *Time Series Model*) у којем је сваки елемент тока директна вредност, односно важи $a_i = A[i]$.
- **Модел касе** (енгл. *Cash Register Model*) у којем елементи тока представљају позитивна ажурирања облика $a_i = (j, I_i)$, где је $I_i \geq 0$, при чему важи

$$A_i[j] = A_{i-1}[j] + I_i.$$

- **Модел окретнице** (енгл. *Turnstile Model*) у којем су дозвољена и позитивна и негативна ажурирања облика $a_i = (j, U_i)$, где је $U_i \in \mathbb{R}$, при чему важи

$$A_i[j] = A_{i-1}[j] + U_i.$$

Овај модел се природно проширује на динамичке графове, који се могу представити као секвенца ажурирања над структуром графа током времена.

Модел временских серија је најједноставнији модел: у њему би сваки елемент тока представљао комплетну структуру графа у одређеном тренутку времена. Овај модел је еквивалентан математичком моделу наведеном у претходном поглављу па неће бити поново разматран.

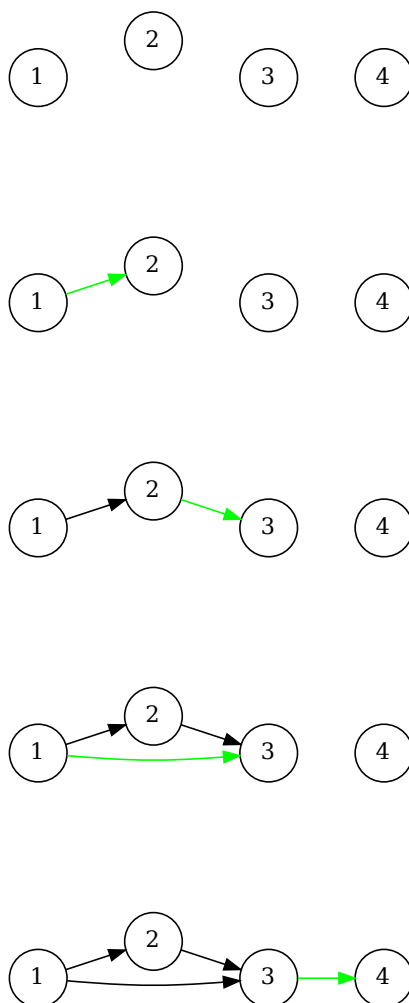
Модел касе и модел окретнице могу се користити за представљање динамичких графова на следећи начин: сваки елемент тока представља додавање или уклањање једног чвора или једне гране у граф. У моделу касе, дозвољена су само позитивна ажурирања, што значи да се чворови и гране могу само додавати у граф, али не и уклањати. У моделу окретнице, дозвољена су и позитивна и негативна ажурирања, што значи да се чворови и гране могу и додавати и уклањати из графа. У наставку ћемо описати оба модела са фокусом на измене грана, док ћемо скуп чворова сматрати константним. Измене чворова могу се посматрати на исти начин као и измене грана.

За потребе модела касе, сматраћемо да је сваки елемент тока уређени пар $e_i = (u, v)$ који представља додавање гране између чворова u и v . Такав ток,

$$S = \langle e_1, e_2, \dots, e_m \rangle$$

дефинише динамички граф $G = (V, E)$ где је V фиксан скуп чворова, а $E = e_1, \dots, e_m$ скуп грана који се мења током времена додавањем нових грана из тока [11]. Такође, елементима тока можемо доделити тежине, тако да сваки елемент тока буде уређени пар $(e_i, w(e_i))$ који представља додавање гране e_i са тежином $w(e_i)$. На тај начин добијамо тежински динамички граф [11]. На слици 2.4 приказан је пример конструкције динамичког графа са четири чвора дефинисан током података $S = \langle (1, 2), (2, 3), (1, 3), (3, 4) \rangle$. Зеленом бојом означене су гране додате у том временском тренутку, а црном бојом означене су гране које су већ биле присутне у графу.

Често системи који се моделују и проблеми који се решавају помоћу динамичких графова не маре за претходна стања графа. Очекивања су да се



Слика 2.4: Динамички граф са четири чвора дефинисан током података $S = \langle (1, 2), (2, 3), (1, 3), (3, 4) \rangle$.

одржавају само информације о тренутном стању графа, али и да су те информације ажурне и да се могу ефикасно обрађивати. Због тога ћемо, за потребе имплементације, у наредном поглављу дефинисати ослабљену верзију динамичког графа, која ће нам омогућити да се фокусирамо на тренутно стање графа и да ефикасно обрађујемо промене које се дешавају током времена.

Глава 3

Имплементациони модел и структура

Иако нам математички модели дају прецизну дефиницију динамичких графова, они нису увек погодни за практичну примену због своје сложености и често се у пракси очекује да знамо тренутно стање графа, без потребе за корацима који су до тог стања довели. Поред тренутног стања пожељно је и да се одржавају одређена својства графа, зависно од потреба проблема. У овом поглављу биће представљен једноставнији модел динамичких графова који је погодан за имплементацију, као и структура података која се користи у имплементацији.

3.1 Ограничења формалног модела и поједностављен модел

Иако математички модел динамичког графа пружа јасну формализацију, његова директна примена у практичним системима има значајна ограничења. Пре свега, овај модел подразумева чување комплетног стања графа за сваки временски тренутак, што доводи до велике просторне сложености, која је реда $O(|T| \cdot (|V| + |E|))$. Поред тога, у одсуству експлицитних информација о ажурирањима, прелаз између узастопних стања захтева поређење комплетних структура или поновно израчунавање својстава графа, што је рачунски неефикасно.

Даље, овакав модел није у складу са начином на који се промене јављају

у реалним системима, где се оне појављују као појединачна ажурирања, а не као комплетне верзије графа. Због тога модели који не подржавају ефикасну обраду ажурирања и инкрементално одржавање својстава графа имају ограничену практичну примену [8].

Због наведених ограничења, у практичним применама се користе поједностављени модели који се фокусирају на одржавање тренутног стања графа и подршку ефикасним ажурирањима.

У наставку се разматра један такав модел, у којем се динамички граф посматра искључиво кроз своје тренутно стање, без експлицитног чувања историје промена. За његову имплементацију користи се структура података *хеш-индексираних блокова суседства* (енгл. *Dynamic Hashed Blocks - DHB*) [14]. Пре него што детаљно обрадимо DHB структуру, биће представљено неколико других поједностављених модела који се користе у литератури, као и разлози за избор DHB структуре у овој имплементацији.

3.2 Постојећи поједностављени модели

Стандардни модели за представљање графова укључују листе суседства и матрице суседства. Међутим, ови модели нису погодни за динамичке графове због неефикасних ажурирања структуре.

У контексту динамичких графова, разматрају се два основна циља. Први циљ је постизање високе брзине ажурирања уз минималну потрошњу меморије. Други циљ је убрзавање аналитичких алгоритама који се извршавају над графом. Већина постојећих приступа фокусира се превасходно на први циљ, занемарујући ефикасност аналитичких операција [14].

У литератури су предложене различите структуре података са бољом ефикасношћу за рад са динамичким графовима, као што су:

- **Компресовани ретки редови** (енгл. *Compressed Sparse Row (CSR)*) - компактна репрезентација са просторном сложеностју $O(|V| + |E|)$. Погодна за статичке графове, али непогодна за честа ажурирања [15, 8].
- **STINGER** (*Spatio-Temporal Interaction Networks and Graphs Extensible Representation*) - динамичка структура заснована на блоковима повезаним у листе, са подршком за паралелно извршавање. Омогућава ефи-

касна ажурирања и анализу великих графова, али је сложенија за имплементацију [5].

- **Хеш индексирани блокови суседства** (енгл. *Dynamic Hashed Blocks (DHB)*) - блоковска структура са хеш индексом који омогућава брз приступ суседима. Представља компромис између ефикасности ажурирања и сложености имплементације [14].

Из наведених структура, CSR није погодан за динамичке графове због скупих ажурирања, док STINGER иако веома ефикасан, уводи значајну имплементациону сложеност. Структура *хеш-индексираних блокова суседства* представља компромис између ова два приступа, задржавајући добре перформансе уз знатно једноставнију имплементацију, због чега је изабрана за даљу анализу и реализацију.

3.3 DHB структура

Структура података *хеш-индексираних блокова суседства* (DHB) је заснована на блоковима меморије са додатним хеш индексом за убрзање приступа суседима и проверу постојања грана. Иако се ослања на једноставне механизме управљања меморијом, експериментални резултати показују да може постићи перформансе упоредиве са савременим структурама за рад са динамичким графовима [14].

DHB је дизајниран тако да су операције важне за рад са динамичким графовима ефикасне. Ове операције укључују [14]:

- Основне операције - DHB даје стандардни интерфејс за рад са графовима који подржава следеће операције: додавање и уклањање чворова и грана, промену тежина грана, проверу постојања грана и итерацију кроз суседе.
- Произвољан распоред суседа - одређени алгоритми захтевају да суседи буду распоређени на одређени начин како би радили исправно (нпр. алгоритам *Suitor*¹). Како би ово било омогућено, структура података

¹*Suitor* је похлепни алгоритам за апроксимацију максималног упаривања који током извршавања користи уређен обилазак суседа. Да би се обезбедило детерминистичко понашање у случају једнаких тежина, уводи се потпуно уређење грана инцидентних истом чвору: ако су гране u,v и u,w исте тежине и $v < w$, онда важи $w(u,v) < w(u,w)$ [1].

не би требало да намеће фиксни распоред суседа. ДНВ захваљујући свом дизајну омогућава произвољно преуређивање суседа, што га чини погодним за алгоритме који захтевају специфичан редослед суседа.

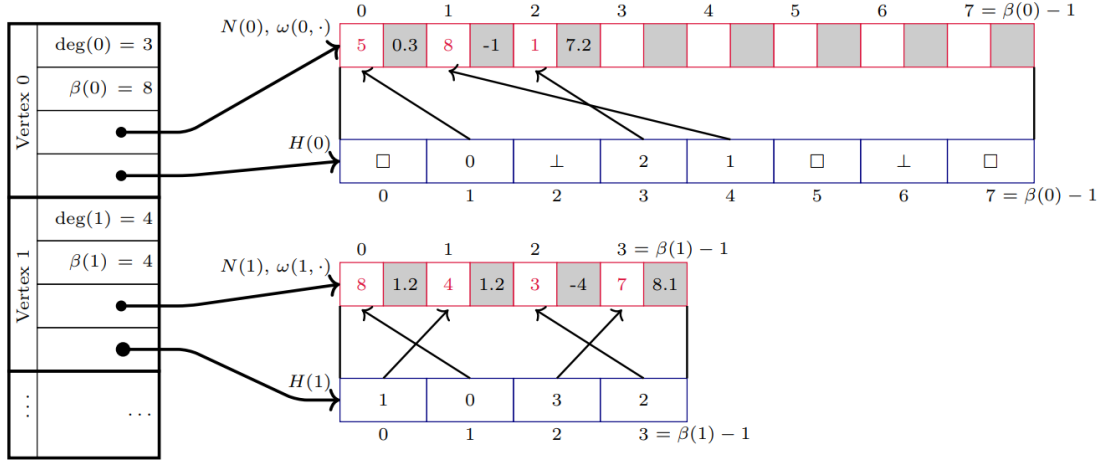
- **Подршка за конкурентно извршавање** - многи алгоритми за рад са динамичким графовима подразумевају паралелну обраду суседа различитих чворова. ДНВ ово омогућава коришћењем појединачних хеш индекса за сваки чвор чиме се омогућава конкурентно извршавање.
- **Насумичан приступ** - многи алгоритми очекују да могу приступити i -том ($i \in [0, \deg(u))$) суседу чвора u у константном времену, на пример када се приступа насумичном суседу. ДНВ подржава овај захтев тако што суседе чува секвенцијално у низу, што омогућава директан индексни приступ.

ДНВ је дизајниран тако да сваки чвор у графу садржи блок у коме су уписани његови суседи. Сваки блок садржи низ суседа као и хеш индекс који омогућава брзу проверу постојања и позицију суседа. Овај дизајн омогућава једноставно управљање и ажурирање суседства, као и ефикасно итерирање кроз суседе. Прецизније, сваки чвор садржи следећа поља [14]:

- **$\deg(u)$** - тренутни степен чвора u , односно број његових суседа.
- **$\beta(u)$** - позитиван цео број који представља тренутну величину блока суседа чвора u .
- **Низ суседа $N(u)$** - низ који садржи суседе чвора u , где првих $\deg(u)$ позиција садржи тренутне суседе, а остатак је празан.
- **Хеш индекс $H(u)$** - структура која омогућава брзу проверу постојања суседа чвора u .

Да би се олакшало одржавање хеш индекса, величина блока суседа $\beta(u)$ је увек степен двојке. Ово омогућава ефикасно управљање блоковима и брзо израчунавање позиције суседа у низу. Такође, увек је гарантовано да је $\deg(u) \leq \beta(u)$. Због тога је у низу суседа првих $\deg(u)$ позиција увек заузето, док су преостале позиције празне и спремне за нове суседе који ће бити додани у будућности [14].

Коначно, граф је представљен као низ хеш-индексираних блокова суседства, где је сваки блок везан за одређени чвор. На слици 3.1 је приказана структура података DHB.



Слика 3.1: Структура хеш-индексираних блокова суседства (DHB) [14]

Имплементација блока суседства у језику C++ представљена је на листингу 3.1. Чвор је представљен класом **Vertex**, која садржи поља за идентификатор чвора, степен, величину блока суседа, низ суседа и хеш индекс. Класа такође садржи методе за управљање суседима, као што су додавање, уклањање и промена тежина грана, као и методе за итерацију кроз суседе.

Методе **AddNeighbor**, **RemoveNeighbor** и **ChangeWeight** омогућавају додавање, уклањање и измену тежина грана. Приликом додавања суседа, нови елемент се уписује на прву слободну позицију у низу суседа, док се хеш индекс ажурира тако да омогући константно време приступа. Уклањање суседа може се реализовати у два режима: без очувања редоследа, када се елемент замењује последњим елементом у низу (што омогућава сложеност $O(1)$), и са очувањем редоследа, када се врши померање елемената (што доводи до сложености $O(\deg(u))$).

Методе **IsNeighbor** и **GetWeight** користе хеш индекс за ефикасну проверу постојања суседа v и приступ тежини гране између u и v , са очекиваном временском сложености $O(1)$. Итерација кроз суседе реализована је преко метода **NeighbourIterator** и **NeighbourEndIterator**, који омогућавају линеарни пролаз кроз првих $\deg(u)$ елемената низа.

Интерне методе `FindSlot`, `FindInsertSlot` и `HashToSlot` реализују основне операције над хеш индексом, укључујући израчунавање почетне позиције и линеарно испитивање слотова приликом решавања колизија (енгл. *linear probing*). Методе `RebuildHashIndex` и `GrowAndRehash` служе за одржавање конзистентности и контролу фактора попуњености, при чему се у случају потребе врши проширивање блока и поновно хеширање елемената.

Додатне методе, као што су `SortEdges` и `RemapNeighborIdsAfterVertex Removal`, омогућавају специфичне трансформације над суседством, при чему је након ових операција неопходно поново изградити хеш индекс.

Детаљна анализа и опис механизма хеш индекса, укључујући стратегију разрешавања колизија и управљање слотовима, дата је у наредној подсекцији.

Листинг 3.1: Интерфејс класе `Vertex`

```
1 class Vertex {
2 private:
3     VertexID id;
4     int degree;
5     int beta;
6     std::vector<Neighbor> neighbors;
7     std::vector<HashEntry> hashes;
8
9     int FindSlot(VertexID neighbor_id) const;
10    int FindInsertSlot(VertexID neighbor_id, bool& already_present) const;
11    int HashToSlot(VertexID neighbor_id) const;
12    void RebuildHashIndex();
13    void GrowAndRehash();
14
15 public:
16     Vertex() = delete;
17     explicit Vertex(VertexID id);
18
19     VertexID Id() const;
20     int Degree() const;
21     int Beta() const;
22
23     void AddNeighbor(VertexID neighbor_id, Weight weight = 1);
24     void ChangeWeight(VertexID neighbor_id, Weight new_weight);
```

```

25     void RemoveNeighbor(VertexID neighbor_id, bool preserve_order =
        false);
26     bool IsNeighbor(VertexID neighbor_id) const;
27     Weight GetWeight(VertexID neighbor_id) const;
28
29     std::vector<Neighbor>::const_iterator NeighbourIterator() const;
30     std::vector<Neighbor>::const_iterator NeighbourEndIterator() const;
31
32     void SortEdges();
33     void SetId(VertexID new_id);
34     void RemapNeighborIdsAfterVertexRemoval(VertexID removed_vertex_id);
35 };

```

Граф је представљен класом **Graph**, која садржи низ чворова и број чворова у графу. Класа пружа методе за управљање чворовима и гранама, као што су додавање и уклањање чворова, додавање и уклањање грана, промена тежина грана, провера постојања грана и итерација кроз суседе. Све методе овог интерфејса представљају само омотач око метода класе **Vertex** са позивима за конкретан чвор. Интерфејс класе **Graph** представљен је на листингу 3.2.

Листинг 3.2: Интерфејс класе **Graph**

```

1 class Graph {
2 public:
3     Graph() = default;
4
5     int VertexCount() const;
6     VertexID AddVertex();
7     void RemoveVertex(VertexID vertex_id);
8
9     void AddEdge(VertexID vertex_id1, VertexID vertex_id2, Weight weight
        = 1);
10    void RemoveEdge(VertexID vertex_id1, VertexID vertex_id2);
11    Weight GetEdgeWeight(VertexID vertex_id1, VertexID vertex_id2) const;
12    void UpdateEdgeWeight(VertexID vertex_id1, VertexID vertex_id2,
        Weight new_weight);
13
14    bool AreNeighbors(VertexID vertex_id1, VertexID vertex_id2) const;

```

```

15     std::vector<Vertex>::const_iterator VertexIterator() const;
16     std::vector<Vertex>::const_iterator VertexEndIterator() const;
17     std::vector<Neighbor>::const_iterator NeighbourIterator(VertexID
    vertex_id) const;
18     std::vector<Neighbor>::const_iterator NeighbourEndIterator(VertexID
    vertex_id) const;
19
20 private:
21     std::vector<Vertex> vertices;
22     int vertex_count = 0;
23
24 };

```

Хеш индекс

Хеш индекс представља кључни механизам DHB структуре који омогућава ефикасну проверу постојања суседа и приступ њиховим позицијама у низу суседа. За сваки чвор u дефинисан је хеш индекс $H(u)$ који садржи $\beta(u)$ елемената и реализован је као хеш табела са отвореним адресирањем² [14].

За разлику од класичних хеш табела, хеш индекс не складишти директно податке о суседима, већ индексе у низу суседа $N(u)$. Сваки слот у хеш индексу може бити у једном од три стања [14]:

- **EMPTY** - слот никада није био заузет,
- **OCCUPIED** - слот садржи важећи индекс у низу суседа,
- **DELETED** - слот је раније био заузет, али је елемент уклоњен (тзв. *tombstone*).

Уколико је $H(u)[j]$ у стању **OCCUPIED**, онда тај елемент садржи индекс i такав да важи $N(u)[i] = v$, где је v сусед чвора u . Важи да је $i \in [0, \deg(u))$, чиме се обезбеђује конзистентност између хеш индекса и низа суседа [14].

На почетку су сви елементи хеш индекса постављени у стање **EMPTY**. Приликом додавања суседа, одговарајући слот се означава као **OCCUPIED** и у њега

²Хеш табела са отвореним адресирањем је структура у којој се сви елементи смештају директно у низ фиксне величине, а колизије се решавају испитивањем других позиција унутар истог низа (нпр. линеарним испитивањем). [4]

се уписује индекс новог суседа у низу $N(u)$. Приликом уклањања суседа, слот се не празни у потпуности, већ се поставља у стање **DELETED**, чиме се омогућава исправно функционисање линеарног испитивања [14].

За одређивање почетне позиције користи се хеш функција $h : V \rightarrow \mathbb{N}$:

$$j = h(v) \bmod \beta(u)$$

Како је $\beta(u)$ степен двојке, ова операција се у имплементацији извршава као:

$$j = h(v) \& (\beta(u) - 1)$$

У случају колизије примењује се линеарно испитивање:

$$j = (j + 1) \& (\beta(u) - 1)$$

Овај поступак се понавља све док се не пронађе одговарајући слот [14].

Претрага суседа v у хеш индексу чвора u врши се тако што се креће од иницијалне позиције $j = h(v) \bmod \beta(u)$ и наставља линеарним испитивањем:

- ако је $H(u)[j] = \text{EMPTY}$, претрага се завршава и v није сусед,
- ако је $H(u)[j] = \text{OCCUPIED}$ и $N(u)[H(u)[j]] = v$, сусед је пронађен,
- у супротном, наставља се испитивање наредног слота.

Приликом уметања новог суседа v , прво се пролази кроз секвенцу линеарног испитивања како би се проверило да ли сусед већ постоји. Током овог процеса памти се први слот у стању **DELETED**. Уколико се наиђе на слот у стању **EMPTY**, уметање се врши у тај слот или у раније пронађени **DELETED** слот. На овај начин се избегавају дупликати и омогућава ефикасно поновно коришћење слотова [14].

Приликом брисања суседа, одговарајући слот у хеш индексу се означава као **DELETED**. Ово је неопходно како би се очувала исправност секвенци линеарног испитивања, јер би постављање слота у стање **EMPTY** могло прекинути претрагу за елементима који су уметнути касније у истој секвенци [14].

Током времена може доћи до акумулације **DELETED** слотова, што негативно утиче на перформансе. Због тога се периодично врши реконструкција хеш индекса (*rehashing*), при чему се сви важећи суседи чвора u изнова убацују у празну хеш табелу. Ова операција има сложеност $O(\deg(u))$, али се дешава ретко и не утиче на амортизовану сложеност основних операција [14].

У конкретној имплементацији, хеш индекс је представљен низом структура `HashEntry`, где сваки елемент садржи:

- индекс у низу суседа,
- стање слота (`HashValue`).

Структура и енумерација приказани су на листингу 3.3. Вредности `EMPTY` и `DELETED` реализоване су као специјалне негативне вредности, док `OCCUPIED` означава валидан унос. Ова репрезентација омогућава једноставну и ефикасну проверу стања слота током извршавања операција.

Листинг 3.3: Елементи хеш индекса

```
1 enum HashValue {
2     OCCUPIED = 0,
3     EMPTY = -1,
4     DELETED = -2
5 };
6
7 struct HashEntry {
8     unsigned index;
9     HashValue hash_value;
10
11     HashEntry() : index(-1), hash_value(EMPTY) {}
12     HashEntry(unsigned index, HashValue hash_value) : index(index),
13         hash_value(hash_value) {}
14 };
```

3.4 Операције и сложености

Како је граф представљен као низ хеш-индексираних блокова суседства, све операције над графом се свде на операције над појединачним блоковима. Интерфејс класе `Graph` представља једноставан омотач који делегира позиве одговарајућим методама класе `Vertex`. Због тога временске сложености операција над графом директно зависе од сложености операција над блоковима суседства.

Основне операције и њихове временске сложености су [14]:

- **Додавање чвора** се извршава у времену $O(1)$, јер подразумева иницијализацију новог блока суседства са празним низом и хеш индексом.
- **Уклањање чвора** има сложеност $O(|V|)$ у најгорем случају, јер је потребно ажурирати информације о суседима свих чворова који су били повезани са уклоњеним чвором.
- **Додавање гране** (u, v) се у општем случају извршава у времену $O(1)$. Прво се путем хеш индекса проверава да ли се чвор v већ јавља као сусед чвора u . Уколико то није случај, елемент се додаје на позицију $\deg(u)$ у низу $N(u)$, након чега се ажурира степен чвора $u - \deg(u)$ и хеш индекс. Уколико је потребно уметање на произвољну позицију уз очување редоследа, потребно је извршити померање елемената, што доводи до сложености $O(\deg(u))$.
- **Уклањање гране** (u, v) се извршава у времену $O(1)$ уколико није потребно очувати редослед суседа, тако што се елемент замењује последњим елементом у низу и затим уклања. Уколико је потребно очувати редослед суседа, долази до померања елемената, што има сложеност $O(\deg(u))$.
- **Промена тежине гране** (u, v) извршава се у времену $O(1)$, јер се позиција суседа проналази преко хеш индекса, након чега се вредност директно ажурира.
- **Провера постојања гране** (u, v) извршава се у очекиваном времену $O(1)$ коришћењем хеш индекса.
- **Итерација кроз суседе** чвора u има сложеност $O(\deg(u))$, јер се врши линеарни пролаз кроз првих $\deg(u)$ елемената низа $N(u)$, без приступа хеш индексу.
- **Промена редоследа суседа** (нпр. сортирање) врши се над низом $N(u)$. Сложеност ове операције зависи од конкретне трансформације која се примењује (нпр. $O(\deg(u) \log \deg(u))$ у случају сортирања). Након измене редоследа неопходно је поново изградити хеш индекс, што је операција сложености $O(\deg(u))$. Укупна сложеност је стога одређена сложеношћу примењене операције увећаном за трошак реконструкције хеш индекса.

Глава 4

Алгоритми

Алгоритми за рад са динамичким графовима имају улогу да ефикасно одржавају информације од важности у графовима чија се структура мења током времена. За разлику од алгоритама над статичким графовима, алгоритми за динамичке графове често, поред саме алгоритамске процедуре, захтевају и употребу специјализованих структура података које омогућавају ефикасно ажурирање и одржавање информација. Због тога су овакви алгоритми по правилу сложенији и концептуално другачији у односу на класичне алгоритме над статичким графовима који се најчешће срећу у литератури.

У овом поглављу разматрају се проблеми одржавања информација о повезаности чворова, одржавања минималног разапињућег стабла и упаривања у динамичким графовима. Ови проблеми имају значајну улогу у теорији графова и бројне примене у различитим областима, укључујући рачунарске мреже, друштвене мреже и биоинформатику.

4.1 Динамичка повезаност

Одржавање информације о повезаности чворова један је од најпроучаванијих проблема у области динамичких графова. Основни упит који је потребан обезбедити је `isConnected(u, v)`, који проверава да ли постоји пут између чворова u и v у неусмереном графу G [9]. У литератури се могу пронаћи ефикасне структуре података које омогућавају одржавање информација о повезаности у динамичким графовима, као што су *Link-Cut Tree*, *Euler Tour Tree* и *Top Tree* [9, 7]. Ове структуре података омогућавају веома ефикасно одржавање информација о повезаности у динамичким графовима, али су по

правилу сложене за имплементацију и специјализоване за ову групу упита и операција над графом.

У овом поглављу биће представљено проширење ДНВ структуре података које омогућава одржавање информација о повезаности у инкременталним неусмереним динамичким графовима. Ово проширење засновано је на структури дисјунктних подскупова (енгл. *union-find*) и омогућава ефикасно одржавање информација о компонентама повезаности у динамичким графовима.

Упит `isConnected(u, v)` се, захваљујући ефикасности дисјунктних подскупова, извршава у амортизованом константном времену без потребе за обиласком графа. Додавање чворова има константу сложеност јер директно користи ДНВ, док је додавање грана $O(\log |V|)$ због потребе за спајањем компоненти повезаности. Ово проширење ДНВ структуре података представља ефикасно решење за одржавање информација о повезаности у инкременталним неусмереним динамичким графовима.

Имплементација

Имплементација класе `DynamicConnectivityIncremental` представља основни интерфејс за одржавање информације о повезаности у динамичком графу у инкременталном сценарију. Топологија графа и тежине грана моделују се структуром ДНВ, док се информације о компонентама повезаности чувају у помоћној структури дисјунктних подскупова. Интерфејс класе `DynamicConnectivityIncremental` приказан је на листингу 4.1.

Основна идеја имплементације је да се свака компонента повезаности графа представи као један дисјунктан подскуп, при чему се операције `find` и `unite` користе за одређивање репрезента компоненте и спајање двеју компоненти након додавања гране. Компресија путање у функцији `find` (која рекурзивно превезује чворове директно за корен стабла), као и рангирано спајање у оквиру методе `unite`, обезбеђују готово константну амортизовану сложеност по операцији, што је кључно за скалабилност структуре у динамичком окружењу. Притом, ДНВ структура генерише секвенцијалне идентификаторе чворова почев од нуле, што омогућава директно индексирање кроз векторе родитеља и рангова.

Одржавање грана имплементирано је у методама `AddEdge`, `GetEdgeWeight` и `UpdateEdgeWeight`, где се операције над тежинама свODE на матичне опера-

ције DNB структуре, уз додатно позивање методе `unite` приликом уметњања нове гране.

Метод `IsConnected` користи се за испитивање повезаности двају чворова тако што проверава да ли они припадају истом дисјунктном подскупу (односно да ли имају истог репрезента). Уколико два чвора припадају истом дисјунктном подскупу имплементација нам гарантује да се они налазе у истој компоненти повезаности. Како је реч о неусмереном графу, овај приступ гарантује коректност решења.

Листинг 4.1: Интерфејс класе `DynamicConnectivityIncremental`

```

1 class DynamicConnectivityIncremental {
2 private:
3     dg::Graph graph;
4     std::vector<int> parent;
5     std::vector<int> rank;
6
7     int find(int x);
8     void unite(int a, int b);
9
10 public:
11     DynamicConnectivityIncremental() = default;
12
13     int VertexCount() const;
14     dg::VertexID AddVertex();
15
16     void AddEdge(dg::VertexID vertex_id1, dg::VertexID vertex_id2,
17 dg::Weight weight = 1);
18     dg::Weight GetEdgeWeight(dg::VertexID vertex_id1, dg::VertexID
19 vertex_id2) const;
20     void UpdateEdgeWeight(dg::VertexID vertex_id1, dg::VertexID
21 vertex_id2, dg::Weight new_weight);
22
23     bool IsConnected(dg::VertexID vertex_id1, dg::VertexID vertex_id2);
24 };

```

4.2 Динамичко минимално разапињуће стабло

Проблем одређивања минималног разапињућег стабла један је од фундаменталних проблема теорије графова. У контексту динамичких графова, овај проблем се трансформише у проблему ефикасног одржавања минималног разапињућег стабла, односно минималне разапињуће шуме. Динамички приступ решавању овог проблема је знатно комплекснији од статичког. Детерминистички алгоритам који су развили Холм, де Лихтенберг и Торуп постиже амортизовано време ажурирања од $O(\log^4 |V|)$ [9].

Ипак, за потребе овог рада развијен је нешто једноставнији алгоритам заснован на Крускаловом алгоритму¹. Основна идеја је да се након сваке измене у динамичком графу минимална разапињућа шума рачуна изнова. Кључно у овој идеји је могућност одржавања сортираног редоследа чворова у блоку суседа које нам даје структура ДНВ. Сложеност ажурирања у овом случају износи $O((|V| + |E|)\log|V|)$. Ипак кључна идеја овде је представити могућност ДНВ структуре да одржава сортирани поредак суседа.

Имплементација

Имплементација инкременталног динамичког алгоритма за одржавање минималне разапињуће шуме представљена је класом `DynamicMSTIncremental`. Подаци о гранама шуме, структури дисјунктних скупова неопходној за проверу циклуса, као и јавне методе за манипулацију графом дефинисани су унутар ове класе. На листингу 4.2 приказан је комплетан интерфејс класе `DynamicMSTIncremental`.

Листинг 4.2: Интерфејс класе `DynamicMSTIncremental`

```

1 class DynamicMSTIncremental {
2 private:
3     struct Edge {
4         dg::VertexID u;
```

¹Крускалов алгоритам је похлепни (*greedy*) алгоритам који проналази минимално разапињуће стабло за повезан тежински граф (или шуму за неповезан граф). Алгоритам ради тако што најпре сортира све гране графа по тежини у растућем поретку, а затим sukcesивно додаје грану по грану у резултат, под условом да новододата грана не затвара циклус са већ изабраним гранама [10].

```

5         dg::VertexID v;
6         dg::Weight w;
7     };
8
9     dg::Graph graph;
10
11     std::vector<Edge> mst_edges;
12
13     std::vector<int> parent;
14     std::vector<int> rank;
15
16     dg::Weight mst_weight = 0;
17
18     int find(int x);
19     bool unite(int a, int b);
20
21     void rebuildMST();
22
23 public:
24     DynamicMSTIncremental() : graph(true), mst_weight(0) {}
25
26     int VertexCount() const;
27     dg::VertexID AddVertex();
28
29     void AddEdge(dg::VertexID vertex_id1,
30                 dg::VertexID vertex_id2,
31                 dg::Weight weight = 1);
32
33     dg::Weight GetEdgeWeight(dg::VertexID vertex_id1,
34                              dg::VertexID vertex_id2) const;
35
36     void UpdateEdgeWeight(dg::VertexID vertex_id1,
37                           dg::VertexID vertex_id2,
38                           dg::Weight new_weight);
39
40     bool IsInMST(dg::VertexID vertex_id1,
41                  dg::VertexID vertex_id2) const;

```



```
42  
43     dg::Weight GetMSTWeight() const;  
44 };
```

Кључни корак одржавања минималног разаципућег стабла лежи у приватној методи `rebuildMST()`, која се позива након сваког додавања нове гране или модификације постојећих тежина. Уместо класичног Крускаловог алгоритма који би прикупио све гране графа у један вектор и извршио комплетно сортирање у времену $O(|E| \log |E|)$, овде се примењује ефикаснији приступ спајања већ сортираних низова уз помоћ мин-хипа (`std::priority_queue`) чиме добијамо сложеност $O(|V| \log |V|)$ што је у општем случају асимптотски ефикасније.

Приликом покретања методе `rebuildMST()`, за сваки чвор графа се дохвата почетни и крајњи итератор кроз његов блок суседа. Пошто су ови суседи захваљујући ДНВ структури већ сортирани по тежини, у мин-хип се иницијално убацује само по прва (најлакша) грана за сваки чвор. У сваком кораку алгоритма, са врха хипа се узима грана најмање тежине, врши се провера и спајање компоненти кроз дисјунктне подскупове, а затим се у хип прослеђује следећа грана тог истог чвора преко његовог итератора. Овим поступком се постиже да величина хипа у сваком тренутку буде ограничена на највише n елемената, што значајно оптимизује перформансе на густим графовима и елиминише додатне алокације меморије на хипу. У сваком кораку алгоритма величина блокова на хипу се смањује што нам гарантује заустављање.

Метод `IsInMST` за дата два чвора проверава да ли се грана између њих налази у стаблу. Метод `GetMSTWeight` даје укупну тежину минималног разаципућег стабла.

4.3 Динамичко максимално тежинско упаривање - Суитор алгоритам

Максимално тежинско упаривање (енгл. *Maximum Weighted Matching*) је упаривање $M \subset E$ у тежинском графу $G = (V, E)$ које има максималну тежину грана [1]. У терминима динамичких графова овај проблем посматрамо као проблем одржавања максималног тежинског упаривања у динамичком

графу. Иако постоје егзактни алгоритми линеарне сложености за решавање овог проблема, у овом раду обрадићемо алгоритам Суитор (енгл. *Suitor*) који израчунава $1/2$ максимално тежинско упаривање, односно даје гаранцију да ће резултат његовог извршавања бити највише за 50% мањи од стварног максималног тежинског упаривања. Разлог за избор овог алгоритма је тај што се овај алгоритам ослања на сортираност грана суседних сваком чвору. Структура DHB може одржавати ово својство што нам даје могућност да то директно користимо у алгоритму.

Статички Суитор алгоритам ослања се да су суседи сваког чвора сортирани по тежини гране опадајуће. Нека је w функција тежине грана. Узмимо, ако $\{u, v\} \notin E$ или ако је референца чвора v нула, онда је $w(u, v) = 0$. Да би се гарантовало коректно извршавање, следећи тотални поредак грана инцидентних истом чвору u се намеће: ако $\{u, x\}$ и $\{u, y\}$ имају исту тежину гране и $x < y$, онда $w(u, x) < w(u, y)$. Суитор чува две референце за сваки чвор $u \in V$, $p(u)$ и $\text{suitor}(u)$, за време извршавања алгоритма. Када је извршавање завршено, тада важи

$$p(u) = \arg \max_{v \in N(u)} \{w(u, v) : x \in N(v), \text{ такво да } p(x) = v \wedge w(x, v) > w(u, v)\}.$$

Другим речима, $p(u)$ је сусед v од u такав да је $w(u, v)$ максимално и не постоји чвор $x \in N(v)$ где је $p(x) = v$ са упаривањем $\{x, v\}$ које доминира над упаривањем $\{u, v\}$. Ако не постоји такав v , тада је $p(u) = \text{null}$, односно неупарен. Обрнуто, $\text{suitor}(v)$ се односи на чвор (ако постоји) који чува референцу на v : $\text{suitor}(u) = v$ ако и само ако је $p(v) = u$ [1].

Иако постоје напредне модификације Суитор алгоритма у динамичку верзију где се одржавање упаривања ради ефикасније таква имплементација излази из оквира овог рада. Идеја коју ћемо овде користити је да DHB структура одржи суседе сваког чвора сортиране по тежини, а да се након сваке измене додатно примени статички Суитор алгоритам на цео граф. Додатна сложеност сваког корака у овом случају је $O(|V|)$ што је сложеност статичког Суитор алгоритма.

Имплементација

Имплементација алгоритма за одржавање максималног тежинског упаривања заокружена је класом `SuitorMatching`. Она служи као драјвер који

енкапсулира DHB граф и помоћне меморијске структуре неопходне за ефикасно проналажење партнера. Комплетан интерфејс класе приказан је на листингу 4.3.

Срж израчунавања упаривања налази се у приватној методи `rebuildMatching()`, која се извршава изнова након сваке структурне модификације графа. Алгоритам користи два пратећа вектора: `suitor`, који за сваки чвор бележи тренутног просиоца (или -1 уколико је слободан), и `suitor_weight`, који чува тежину тренутне понуде.

Процес је додатно оптимизован имплементацијом помоћног вектора `active_vertices`, који функционише као стек за чување тренутно активних чворова који немају партнера, а још увек поседују потенцијалне суседе за прошење. На почетку извршења методе, сви чворови графа се проглашавају активним и смештају се на стек.

Алгоритам користи својство DHB структуре података. С обзиром на то да је граф иницијализован тако да одржава суседе у растућем поретку по тежини, најбољи сусед (онај са највећом тежином гране) се природно налази на самом крају блока суседа. Због тога се у методи `rebuildMatching()` користе реверзни итератори `std::reverse_iterator`. Приступом итератору на крај блока суседства алгоритам у константном времену $O(1)$ приступа суседу са тренутно највећом тежином, што елиминише потребу за експлицитним претраживањем или накнадним сортирањем.

Унутар главне петље, активни чвор шаље понуду свом најбољем суседу. Уколико је тежина те понуде већа од оне коју сусед већ има забележену у `suitor_weight`, сусед прихвата нову понуду. Претходни просилац тог суседа (уколико је постојао) тада бива одбијен и поново се враћа на стек `active_vertices` како би у наредним итерацијама покушао да запроси свог следећег најбољег суседа. Овај процес се детерминистички зауставља када стек постане празан.

Након што се процес прошења стабилизује, врши се финално прикупљање грана које задовољавају услов обостраног суседства ($p(v) = u \wedge p(u) = v$). Те гране се смештају у вектор `matching_edges`, а њихове тежине се акумулирају у `total_matching_weight`. Методе `IsMatched` и `GetPartner` омогућавају брзу проверу статуса и идентификацију партнера за било који чвор у константном времену, једноставним приступом преко индекса вектора.

Листинг 4.3: Интерфейс класе SuitorMatching

```

1 class SuitorMatching {
2 private:
3     dg::Graph graph;
4     std::vector<dg::VertexID> suitor;
5     std::vector<dg::Weight> suitor_weight;
6     std::vector<std::pair<dg::VertexID, dg::VertexID>> matching_edges;
7     dg::Weight total_matching_weight;
8
9     void rebuildMatching();
10
11 public:
12     SuitorMatching() : graph(true), total_matching_weight(0) {}
13
14     dg::VertexID AddVertex();
15     void AddEdge(dg::VertexID vertex_id1,
16                 dg::VertexID vertex_id2,
17                 dg::Weight weight = 1);
18     void RemoveEdge(dg::VertexID vertex_id1,
19                    dg::VertexID vertex_id2);
20     void RemoveVertex(dg::VertexID vertex_id);
21     void UpdateEdgeWeight(dg::VertexID vertex_id1,
22                           dg::VertexID vertex_id2,
23                           dg::Weight new_weight);
24
25     const std::vector<std::pair<dg::VertexID, dg::VertexID>>&
26     GetMatchingEdges() const;
27     dg::Weight GetTotalMatchingWeight() const;
28     bool IsMatched(dg::VertexID vertex_id) const;
29     dg::VertexID GetPartner(dg::VertexID vertex_id) const;
30 };

```

Глава 5

Примене динамичких графова кроз примере

Глава 6

Закључак

Библиографија

- [1] Eugenio Angriman, Michał Boroń, and Henning Meyerhenke. A Batch-dynamic Suitor Algorithm for Approximating Maximum Weighted Matching. *ACM J. Exp. Algorithmics*, 27:1.6:1–1.6:41, July 2022.
- [2] Claudio D. T. Barros, Matheus R. F. Mendonça, Alex B. Vieira, and Artur Ziviani. A survey on embedding dynamic graphs. *ACM Comput. Surv.*, 55(1), November 2021.
- [3] Zi Chen, Keke Liang, Long Yuan, Wenjie Zhang, and Zhengyi Yang. Recent advances in efficient dynamic graph processing. *Applied Sciences*, 15(11), 2025.
- [4] Thomas H. Cormen, editor. *Introduction to Algorithms*. MIT Press, Cambridge, Mass., 3. ed edition, 2009.
- [5] David Ediger, Rob McColl, Jason Riedy, and David A Bader. Stinger: High performance data structure for streaming graphs. In *2012 IEEE Conference on High Performance Extreme Computing*, pages 1–5. IEEE, 2012.
- [6] F. Harary and G. Gupta. Dynamic graph models. *Mathematical and Computer Modelling*, 25(7):79–87, 1997.
- [7] Jacob Holm, Kristian de Lichtenberg, and Mikkel Thorup. Poly-logarithmic deterministic fully-dynamic algorithms for connectivity, minimum spanning tree, 2-edge, and biconnectivity. *J. ACM*, 48(4):723–760, July 2001.
- [8] Abdullah Al Raqibul Islam and Dong Dai. Dgap: Efficient dynamic graph analysis on persistent memory, 2024.
- [9] Ms. Navjot Kaur. Applications of dynamic graph algorithms in real-time traffic and network optimization 2025. *Lex localis - Journal of Local Self-Government*, 23(S4):3073–3087, Aug. 2025.

- [10] Joseph B. Kruskal. On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem. 7(1):48–50.
- [11] Andrew McGregor. Graph stream algorithms: a survey. *SIGMOD Rec.*, 43(1):9–20, May 2014.
- [12] S. Muthukrishnan. *Data Streams: Algorithms and Applications*. Foundations and trends in theoretical computer science. Now Publishers, 2005.
- [13] Zoran Stanić. *Diskretne strukture 2 – osnovi kombinatorike, teorije brojeva i teorije grafova*. Matematički fakultet, Beograd, 2 edition, 2020.
- [14] Alexander van der Grinten, Maria Predari, and Florian Willich. A Fast Data Structure for Dynamic Graphs Based on Hash-Indexed Adjacency Blocks. In Christian Schulz and Bora Uçar, editors, *20th International Symposium on Experimental Algorithms (SEA 2022)*, volume 233 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 11:1–11:18, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [15] Brian Wheatman and Helen Xu. Packed compressed sparse row: A dynamic graph representation. In *2018 IEEE High Performance extreme Computing Conference (HPEC)*, pages 1–7. IEEE, 2018.

Биографија аутора

Марко Лазаревић рођен је 4. априла 2002. године у Фочи, Босна и Херцеговина. Основне студије завршио је 2025. године на смеру Информатика на Математичком факултету Универзитета у Београду, са просечном оценом 9.35.